

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010

OPERATING SYSTEM

WEEK 4 LECTURE 1

MEMORY ADDRESSING

SEGMENT SELECTORS

Memory Addressing

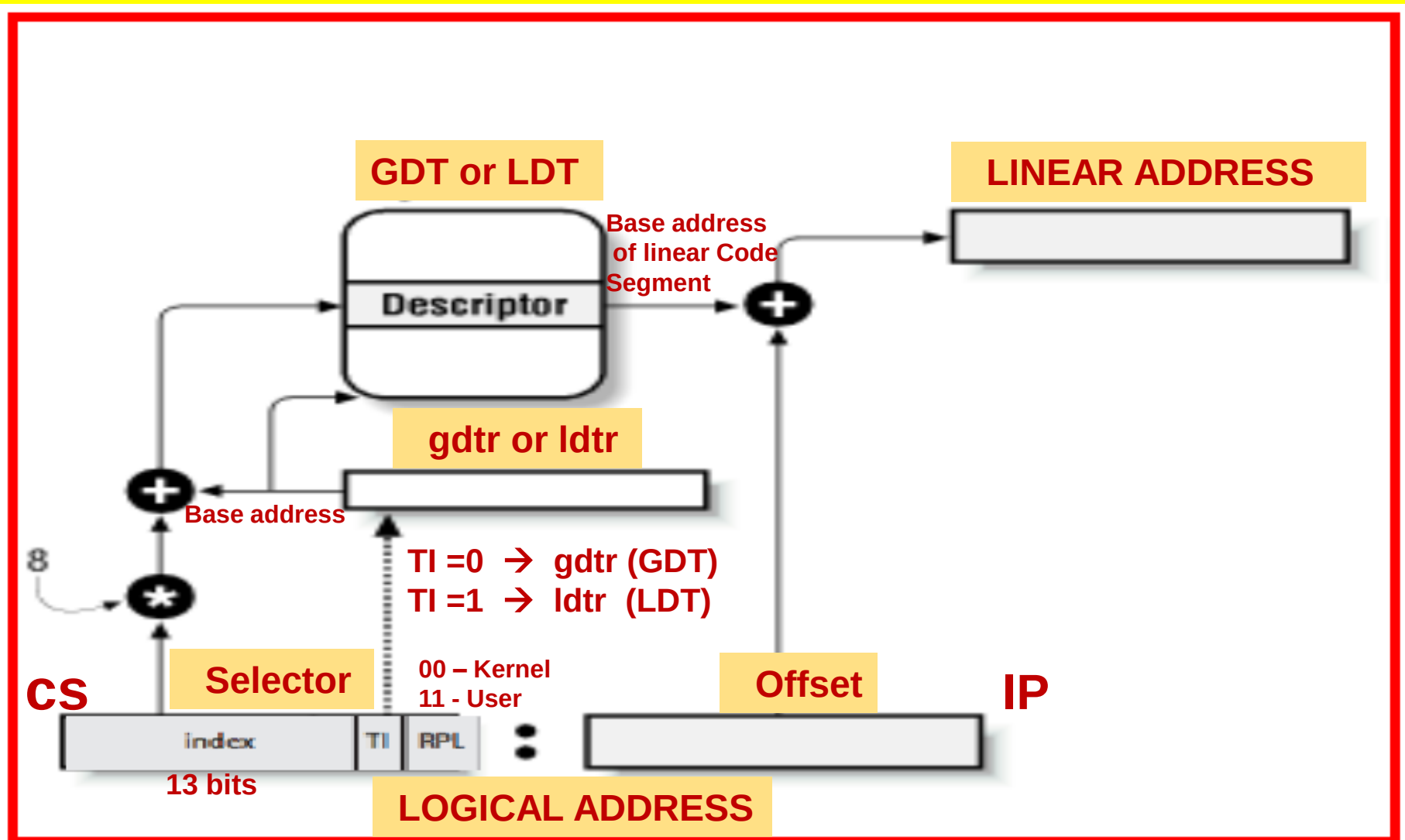
□ Segment Selectors

- ★ Each Segment Selector includes the following fields:
 - A **13-bit index** that identifies the corresponding Segment Descriptor entry contained in the GDT or in the LDT.
 - A **TI (Table Indicator)** flag that specifies whether the Segment Descriptor is included in the GDT (TI = 0) or in the LDT (TI = 1).
 - An **RPL (Requestor Privilege Level)** 2-bit field, which is precisely the Current Privilege Level of the CPU when the corresponding Segment Selector is loaded into the cs register.
- ★ The RPL field may also be used to selectively weaken the processor privilege level when accessing data segments.



Memory Addressing

Segmentation Unit - Translating a logical address to a Linear Address



Memory Addressing

Example of a Segmentation Unit

Given GDT is at 0x90000000 (the value stored in the gdtr register) and

LOGICAL ADDRESS

CS= 0x19B IP=0x100

CS= 0000000110011011

INDEX=0000000110011=0x33

Address of the corresponding Segment Descriptor is

= address in gdtr + INDEX * 8

= 0x90000000 + (0x33 x 8)

= 0x90000000 + 0x198

= 0x90000198

$$\begin{aligned} 0x33 * 8 &= 51 * 8 \\ &= 408 = 0x198 \end{aligned}$$

SEGMENT DESCRIPTOR is at 0x90000198 in GDT

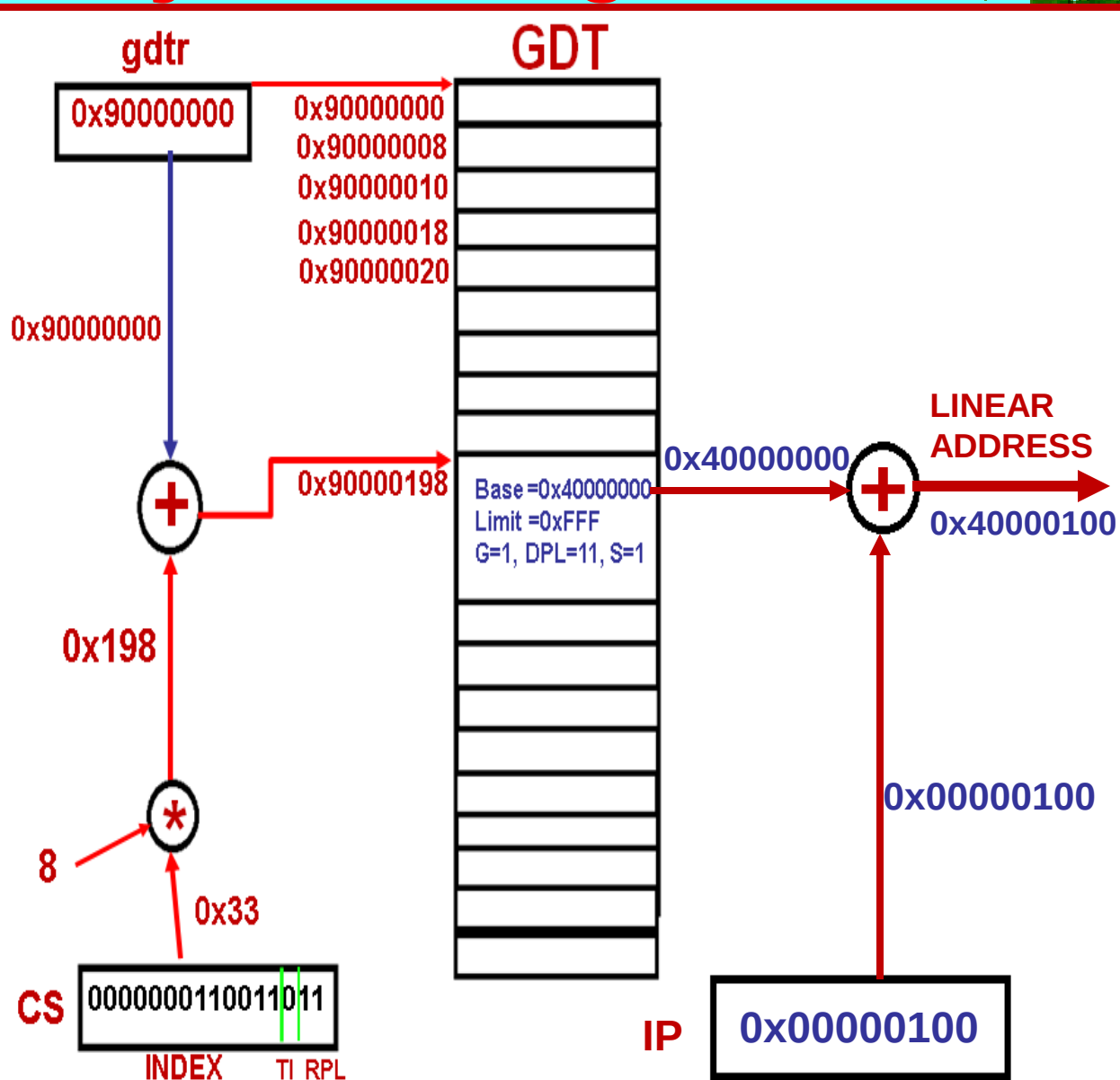
BASE = 0x40000000

LINEAR ADDRESS=BASE + IP

= 0x40000000 + 0x100

LINEAR ADDRESS

0x40000100



Memory Addressing

□ Segment Selectors

- ★ Since a Segment Descriptor is 8 bytes long, its relative address inside the GDT or the LDT is obtained by multiplying the most significant 13-bit index field of the Segment Selector by 8.
- ★ For instance,
if the GDT is at 0x00020000 (the value stored in the gdtr register) and the index specified by the Segment Selector is 2, then the address of the corresponding Segment Descriptor is
$$= \text{Base address in gdtr} + \text{index} * 8 = 0x00020000 + (2 \times 8)$$
$$= 0x00020010.$$
- ★ The first entry of the GDT is always set to 0: this ensures that logical addresses with a null Segment Selector will be considered invalid, thus causing a processor exception.
- ★ The maximum number of Segment Descriptors that can be stored in the GDT is thus 8191, that is, $2^{13} - 1$.

Memory Addressing

QUESTION : Consider the GDT in the figure showing the Segment Descriptors with their starting addresses. If the logical address generated by the processor is given as follows :

CS register contains the value 0xC3 and IP register has the value 0x500 then answer the following. Show all the computations.

Given CS = 0xC3 = 0000000011000011, INDEX = 0000000011000 = 0x18
TI = 0 → access GDT, RPL = 11 → user mode

A) What is the starting address of the corresponding Segment Descriptor stored in the GDT (i.e., address of the corresponding GDT entry).

Starting address of GDT(gdtr) + INDEX * 8

= 0x00A00000 + 0x18 * 8

= 0x00A00000 + 0xC0

= 0X00A000C0

0x18 * 8 = 24 * 8
= 192 = 0xC0

B) What is the base address of the linear segment found in that segment descriptor entry

Base address = 0x30000000.

C) Compute the corresponding Linear Address

Base address + offset in IP

= 0x30000000 + 0x500 = 0x30000500

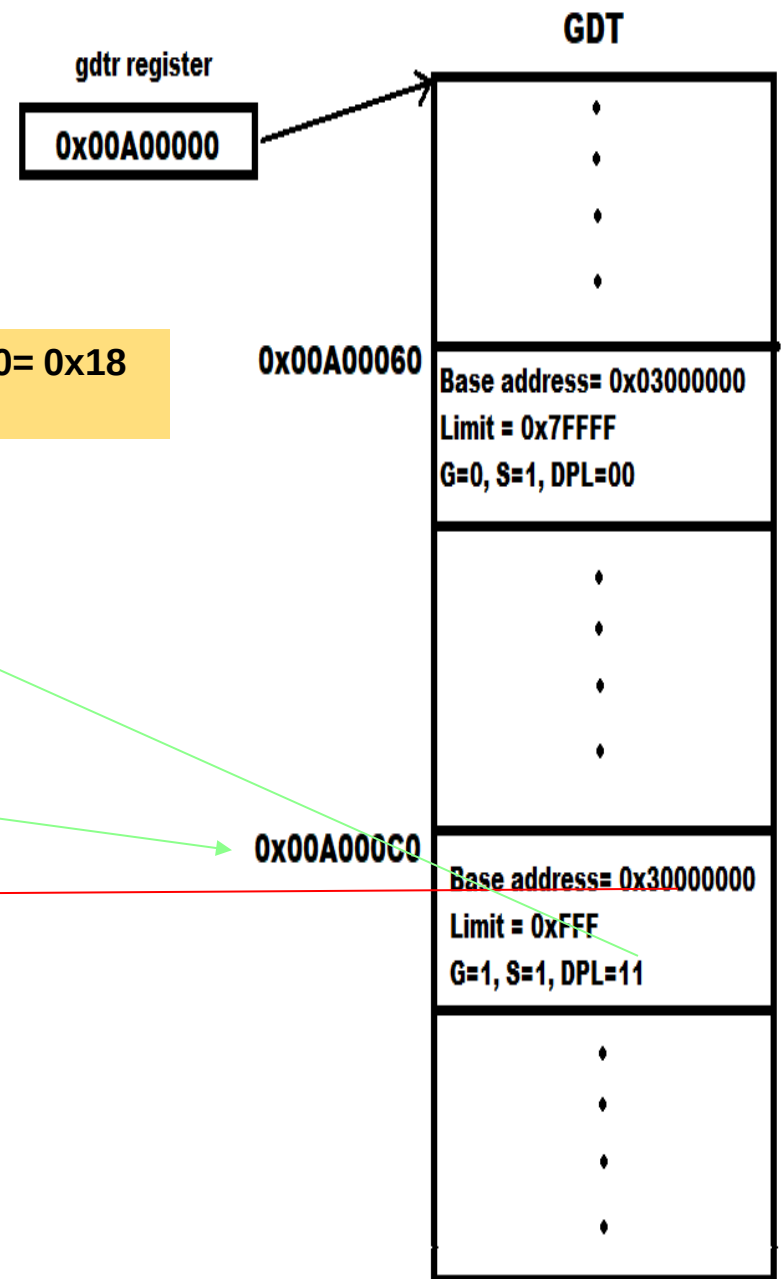
D) Determine the length or size of the Segment

limit = 0xFFF = 2^{12} , G = 1, Segment size = 2^{12} pages = 16MB

E) Compute the ending address of the linear segment

Segment size = 16MB = 2^{24} → limit or offset = FFFFFFFF

Ending address = Starting address + limit = 30FFFFFFF



Memory Addressing

Question : Consider the GDT in the figure below showing the Segment Descriptors with their starting addresses.

If the logical address generated by the processor is given as follows :

CS register contains the value 0x63 and IP register has the value 0x300 then answer the following. Show all the computations.

Given CS = 0x63 = 0000000001100011, INDEX = 0000000001100 = 0x0C
TI = 0 → access GDT, RPL = 11 → user mode

A] What is the starting address of the corresponding Segment Descriptor stored in the GDT (i. e, address of the corresponding GDT entry).

Starting address of GDT(gdtr) + INDEX * 8

= 0x00A00000 + 0x0C * 8 = 0x00A00000 + 0x60

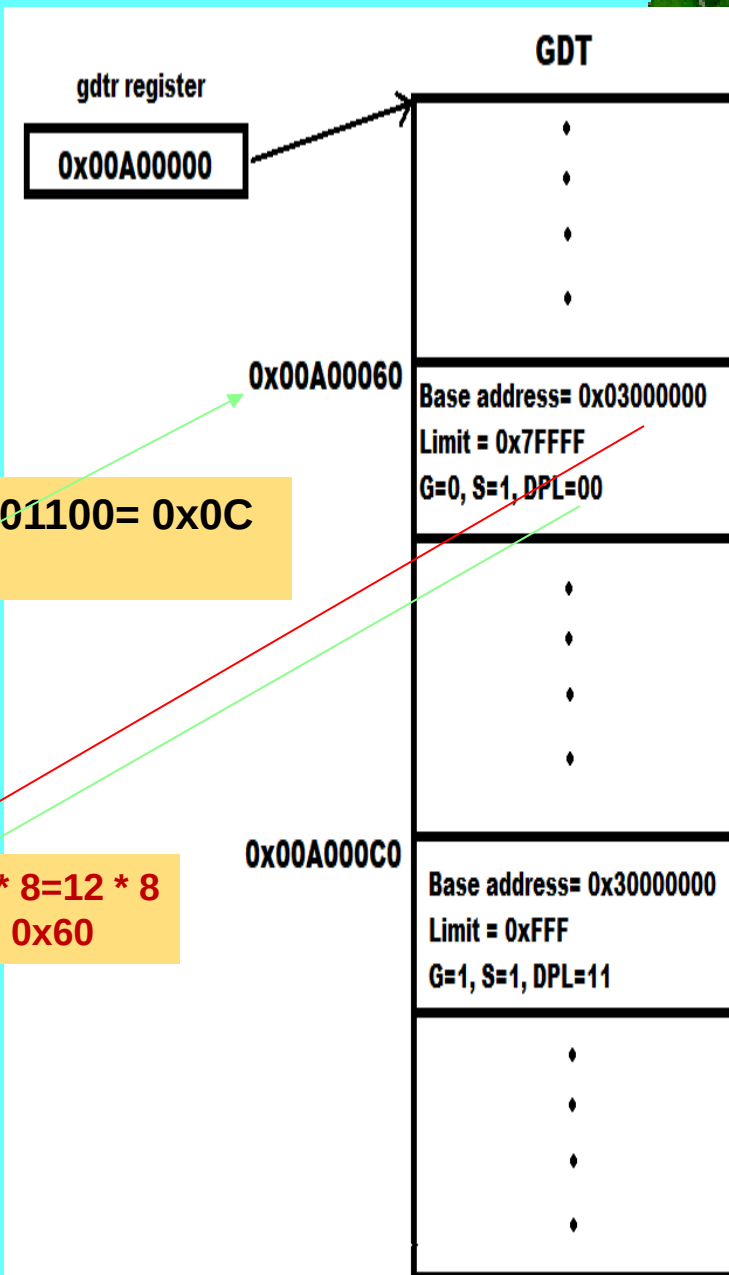
= 0x00A00060

B] What is the base address of the linear segment found in that segment descriptor entry

Base address = 0x03000000

C) Compute the corresponding linear address.

Since DPL = 00, & RPL = 11 → In user mode cannot access the kernel code segment



Memory Addressing

□ Segmentation Unit

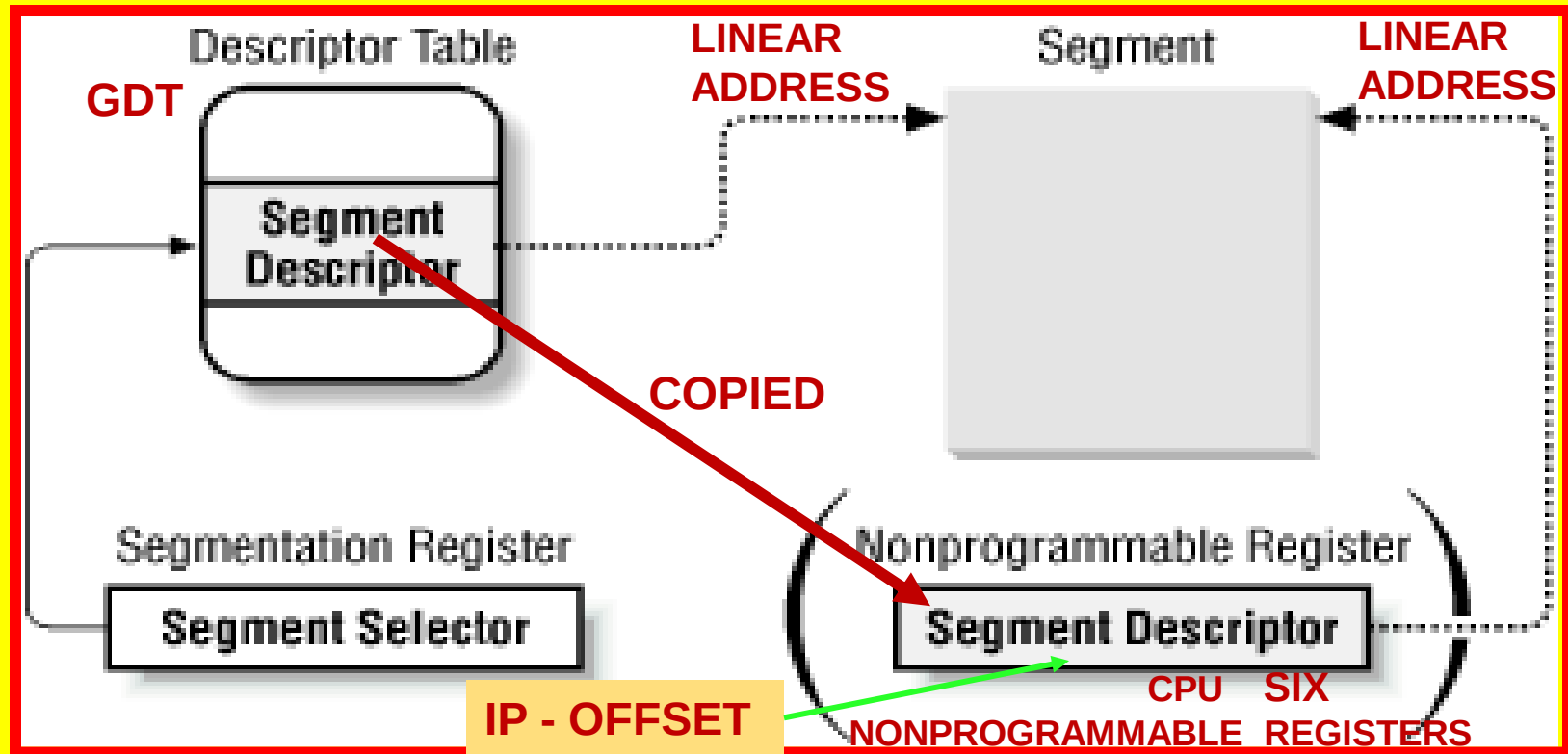
- ✴ Let us look at how a logical address is translated into a corresponding linear address.
- ✴ The segmentation unit performs the following operations:
 - Examines the TI field of the Segment Selector, in order to determine which Descriptor Table stores the Segment Descriptor. This field indicates that the Descriptor is either in the GDT (in which case the segmentation unit gets the base linear address of the GDT from the gdtr register) or in the active LDT (in which case the segmentation unit gets the base linear address of that LDT from the ldtr register).
 - Computes the address of the Segment Descriptor from the index field of the Segment Selector. The index field is multiplied by 8 (the size of a Segment Descriptor), and the result is added to the content of the gdtr or ldtr register.
 - Adds to the Base field of the Segment Descriptor the offset of the logical address, thus obtains the linear address.

Memory Addressing

- **Speeding up the translation of logical addresses into linear addresses**
- ★ **To speed up the translation of logical addresses into linear addresses, the Intel processor provides an additional nonprogrammable register—that is, a register that cannot be set by a programmer—for each of the six programmable segmentation registers.**
- ★ **Each nonprogrammable register contains the 8-byte Segment Descriptor specified by the Segment Selector contained in the corresponding segmentation register.**
- ★ **Every time a Segment Selector is loaded in a segmentation register, the corresponding Segment Descriptor is loaded from memory into the matching nonprogrammable CPU register.**
- ★ **From then on, translations of logical addresses referring to that segment can be performed without accessing the GDT or LDT stored in main memory; the processor can just refer directly to the CPU register containing the Segment Descriptor.**
- ★ **Accesses to the GDT or LDT are necessary only when the contents of the segmentation register change.**

Memory Addressing

★ Segment Selector and Segment Descriptor



Each nonprogrammable register contains the 8-byte Segment Descriptor specified by the Segment Selector contained in the corresponding segmentation register. Every time a Segment Selector is loaded in a segmentation register, the corresponding Segment Descriptor is loaded from memory into the matching nonprogrammable CPU register.

Memory Addressing

❑ Segmentation in Linux

- * Segmentation has been included in Intel microprocessors to encourage programmers to split their applications in logically related entities, such as subroutines or global and local data areas.
- * However, Linux uses segmentation in a very limited way.
- * In fact, segmentation and paging are somewhat redundant since both can be used to separate the physical address spaces of processes:
 - segmentation can assign a different linear address space to each process
 - while paging can map the same linear address space into different physical address spaces.
- * Linux prefers paging to segmentation for the following reasons:
 - Memory management is simpler when all processes use the same segment register values, that is, when they share the same set of linear addresses.
 - One of the design objectives of Linux is portability to the most popular architectures; however, several RISC processors support segmentation in a very limited way.

Memory Addressing

❑ Segmentation in Linux

- ★ The Linux uses segmentation only when required by the Intel 80x86 architecture.
- ★ In particular, all processes use the same logical addresses, so the total number of segments to be defined is quite limited and it is possible to store all Segment Descriptors in the Global Descriptor Table (GDT).
- ★ This table is implemented by the array `gdt_table` referred by the `gdt` variable.
- ★ If you look in the Source Code Index, you can see that these symbols are defined in the file `arch/i386/kernel/head.S`.
- ★ Local Descriptor Tables are not used by the kernel, although a system call exists that allows processes to create their own LDTs.
- ★ This turns out to be useful to applications such as Wine (“WINE Is Not windows Emulator”) that execute segment-oriented Microsoft Windows applications.

MEMORY ADDRESSING - SEGMENTATION UNIT

❏ Questions

- * Explain the technique used to speed up the translation of logical addresses into linear addresses.
- * To speed up the translation of logical addresses into linear addresses, the Intel processor provides Registers.
- * What are Non-programmable descriptor registers in intel processor? Why are they required?
- * Theregister in the processor provides the base address of the Global Descriptor Table in the main memory.
- * Theregister in the processor provides the base address of the Local Descriptor Table in the main memory.
- * What are System and Non-System Segments?
- * Describe the following Segment Descriptors. Clearly indicate whether they are System Segments or Non-Segment Segments. Also specify whether they appear only in GDT or only in LDT or both in GDT & LDT
 - a) Code Segment Descriptor
 - b) Data Segment Descriptor
 - c) Task State Segment Descriptor (TSSD)
 - d) Local Descriptor Table Descriptor (LDTD)
- * Explain the technique used to speed up the translation of logical addresses into linear addresses.
- * To speed up the translation of logical addresses into linear addresses, the Intel processor provides Registers.

MEMORY ADDRESSING - SEGMENTATION UNIT

❑ Questions

- * What are Non-programmable descriptor registers in intel processor? Why are they required?
- * Theregister in the processor provides the base address of the Global Descriptor Table in the main memory.
- * Theregister in the processor provides the base address of the Local Descriptor Table in the main memory.
- * The offset of the logical address must be :
 - a) greater than segment limit
 - b) between 0 and segment limit
 - c) between 0 and the segment number
 - d) greater than the segment number
- * In multiprocessor systems, there is GDT for every CPU in the system
 - a) 32
 - b) 16
 - c) 8
 - d) 1
- * Size of each Segment Descriptor stored in GDT or LDT is bytes
 - a. 64 bytes
 - b. 32 bytes
 - c. 16 bytes
 - d. 8 bytes
- * State the purpose of Task State Segment (TSS)
- * In segmentation, each linear address is specified by :
 - a) a segment base address and offset
 - b) a segment selector
 - c) a descriptor base value
 - d) a key
- * The segment base contains the :
 - a) starting linear address of the process
 - b) starting physical address of the segment in memory
 - c) segment length
 - d) None of these

SOC 3010

OPERATING SYSTEM

MEMORY ADDRESSING

GLOBAL DESCRIPTOR TABLE (GDT)

LINUX GDT (GLOBAL DESCRIPTOR TABLE)

- ❑ LINUX GDT (GLOBAL DESCRIPTOR TABLE)
- ★ In uniprocessor systems there is only one GDT, while in multiprocessor systems there is one GDT for every CPU in the system.
- ★ All GDTs are stored in the `cpu_gdt_table` array, while the addresses and sizes of the GDTs (used when initializing the `gdtr` registers) are stored in the `cpu_gdt_descr` array.
- ★ If you look in the Source Code Index, you can see that these symbols are defined in the file `arch/i386/kernel/head.S`.
- ★ Each GDT includes 18 segment descriptors and 14 null, unused, or reserved entries.
- ★ Unused entries are inserted on purpose so that Segment Descriptors usually accessed together are kept in the same 32-byte line of the hardware cache .

LINUX GDT (GLOBAL DESCRIPTOR TABLE)

- ★ The 18 segment descriptors included in each GDT point to the following segments:
- Four user and kernel code and data segments
- A Task State Segment (TSS), different for each processor in the system.
- A segment including the default Local Descriptor Table (LDT), usually shared by all processes
- Three Thread-Local Storage (TLS) segments: this is a mechanism that allows multithreaded applications to make use of up to three segments containing data local to each thread
- Three segments related to Advanced Power Management (APM)
- Five segments related to Plug and Play (PnP) BIOS services
- A special TSS segment used by the kernel to handle "Double fault " exceptions

LINUX GDT (GLOBAL DESCRIPTOR TABLE)

GLOBAL DESCRIPTOR TABLE(GDT)

Linux's GDT	Segment Selectors	Linux's GDT	Segment Selectors
null	0x0	TSS	0x80
reserved		LDT	0x88
reserved		PNPBIOS 32-bit code	0x90
reserved		PNPBIOS 16-bit code	0x98
not used		PNPBIOS 16-bit data	0xa0
not used		PNPBIOS 16-bit data	0xa8
TLS#1	0x30	PNPBIOS 16-bit data	0xb0
TLS#2	0x38	APMBIOS 32-bit code	0xb8
TLS#3	0x40	APMBIOS 16-bit code	0xc0
reserved		APMBIOS data	0xc8
reserved		not used	
reserved		not used	
Kernel Code	0x60 (__KERNEL_CS)	not used	
Kernel Data	0x68 (__KERNEL_DS)	not used	
User Code	0x70 (__USER_CS)	not used	
User Data	0x78 (__USER_DS)	Double Fault TSS	0xf8

LINUX GDT (GLOBAL DESCRIPTOR TABLE)

□ User & Kernel Mode Code & Data Segments

- * All Linux processes running in User Mode use the same pair of segments to address instructions and data.
- * These segments are called user code segment and user data segment , respectively.
- * Similarly, all Linux processes running in Kernel Mode use the same pair of segments to address instructions and data: they are called kernel code segment and kernel data segment , respectively.
- * Table shows the values of the Segment Descriptor fields for these four crucial segments.

Segment	Base	G	Limit	S	Type	DPL	D/B	P
USER CODE	0X00000000	1	0xFFFFFFFF	1	10	3	1	1
USER DATA	0X00000000	1	0xFFFFFFFF	1	2	3	1	1
KERNEL CODE	0X00000000	1	0xFFFFFFFF	1	10	0	1	1
KERNEL DATA	0X00000000	1	0xFFFFFFFF	1	2	0	1	1

- * The corresponding Segment Selectors are defined by the macros `__USER_CS`, `__USER_DS`, `__KERNEL_CS`, and `__KERNEL_DS`, respectively. To address the kernel code segment, for instance, the kernel just loads the value yielded by the `__KERNEL_CS` macro into the CS segmentation register.

LINUX GDT (GLOBAL DESCRIPTOR TABLE)

□ Here are the segments used by Linux:

- **A Kernel code segment**

The fields of the corresponding Segment Descriptor in the GDT have the following values:

- o Base = 0x00000000
- o Limit = 0xFFFFF
- o G (granularity flag) = 1, for segment size expressed in pages
- o S (system flag) = 1, for normal code or data segment
- o Type = 0xa, for code segment that can be read and executed
- o DPL (Descriptor Privilege Level) = 0, for Kernel Mode
- o D/B (32-bit address flag) = 1, for 32-bit offset addresses

Thus, the linear addresses associated with that segment start at and reach the addressing limit of $2^{32}-1$. The S and Type fields specify that the segment is a code segment that can be read and executed. Its DPL value is 0, thus it can be accessed only in Kernel Mode. The corresponding Segment Selector is defined by the `__KERNEL_CS` macro: in order to address the segment, the kernel just loads the value yielded by the macro into the cs register.

LINUX GDT (GLOBAL DESCRIPTOR TABLE)

- **A Kernel data segment**

The fields of the corresponding Segment Descriptor in the GDT have the following values:

- o **Base = 0x00000000**
- o **Limit = 0xffff**
- o **G (granularity flag) = 1**, for segment size expressed in pages
- o **S (system flag) = 1**, for normal code or data segment
- o **Type = 2**, for data segment that can be read and written
- o **DPL (Descriptor Privilege Level) = 0**, for Kernel Mode
- o **D/B (32-bit address flag) = 1**, for 32-bit offset addresses

This segment is identical to the code segment (in fact, they overlap in the linear address space) except for the value of the Type field, which specifies that it is a data segment that can be read and written. The corresponding Segment Selector is defined by the `__KERNEL_DS` macro.

LINUX GDT (GLOBAL DESCRIPTOR TABLE)

- **A user code segment shared by all processes in User Mode**

The fields of the corresponding Segment Descriptor in the GDT have the following values:

- o **Base = 0x00000000**
- o **Limit = 0xffff**
- o **G (granularity flag) = 1**, for segment size expressed in pages
- o **S (system flag) = 1**, for normal code or data segment
- o **Type = 0xa**, for code segment that can be read and executed
- o **DPL (Descriptor Privilege Level) = 3**, for User Mode
- o **D/B (32-bit address flag) = 1**, for 32-bit offset addresses

The S and DPL fields specify that the segment is not a system segment and that its privilege level is equal to 3; it can thus be accessed both in Kernel Mode and in User Mode. The corresponding Segment Selector is defined by the `__USER_CS` macro.

LINUX GDT (GLOBAL DESCRIPTOR TABLE)

- ★ **A user data segment shared by all processes in User Mode**
- ★ **The fields of the corresponding Segment Descriptor in the GDT have the following values:**
 - o **Base = 0x00000000**
 - o **Limit = 0xffff**
 - o **G (granularity flag) = 1, for segment size expressed in pages**
 - o **S (system flag) = 1, for normal code or data segment**
 - o **Type = 2, for data segment that can be read and written**
 - o **DPL (Descriptor Privilege Level) = 3, for User Mode**
 - o **D/B (32-bit address flag) = 1, for 32-bit offset addresses**
- ★ **This segment overlaps the user code segment one: they are identical, except for the value of Type. The corresponding Segment Selector is defined by the `__USER_DS` macro.**

LINUX GDT (GLOBAL DESCRIPTOR TABLE)

- A Task State Segment (TSS) different for each processor in the system
- * The linear address space corresponding to a TSS is a small subset of the linear address space corresponding to the kernel data segment.
- * The Task State Segments are sequentially stored in the `init_tss` array; in particular, the Base field of the TSS descriptor for the `n`th CPU points to the `n`th component of the `init_tss` array.
- * The Base field of the TSS descriptor associated with each process contains the address of the `tss` field of the corresponding process descriptor.
- * The G flag is cleared, while the Limit field is set to `0xeb`, since the TSS segment is 236 bytes long.
- * The Type field is set to 9 or 11 (available 32-bit TSS), and the
- * DPL is set to 0, since processes in User Mode are not allowed to access TSS segments.

Linux LDT(Local Descriptor Table)

- * Most Linux User Mode applications do not make use of a Local Descriptor Table, thus the kernel defines a default LDT to be shared by most processes.
- * This segment is stored in the `default_ldt` variable.
- * The default LDT includes a single entry consisting of a null Segment Descriptor.
- * Each process has its own LDT Segment Descriptor, which usually points to the common default LDT segment.
- * The Base field is set to the address of `default_ldt` and the Limit field is set to 7.
- * If a process requires a real LDT, a new 4096-byte segment is created (it can include up to 511 Segment Descriptors), and the
- * Default LDT Segment Descriptor associated with that process is replaced in the GDT with a new descriptor with specific values for the Base and Limit fields.
- * The `modify_ldt()` system call allows a process to set up its own LDT

LINUX GDT (GLOBAL DESCRIPTOR TABLE)

- ★ The kernel data segment includes a process descriptor for each process.
- ★ Each process descriptor includes its own TSS segment and a pointer to its LDT segment, which is also located inside the kernel data segment.
- ★ Current Privilege Level of the CPU reflects whether the processor is in User or Kernel Mode and is specified by the RPL field of the Segment Selector stored in the CS register.
- ★ Whenever the Current Privilege Level is changed, some segmentation registers must be correspondingly updated.
- ★ For instance, when the CPL is equal to 3 (User Mode), the DS register must contain the Segment Selector of the user data segment, but when the CPL is equal to 0, the DS register must contain the Segment Selector of the kernel data segment.
- ★ A similar situation occurs for the SS register: it must refer to a User Mode stack inside the user data segment when the CPL is 3, and it must refer to a Kernel Mode stack inside the kernel data segment when the CPL is 0.
- ★ When switching from User Mode to Kernel Mode, Linux always makes sure that the SS register contains the Segment Selector of the kernel data segment.

LINUX GDT (GLOBAL DESCRIPTOR TABLE)

- ★ The TSS and LDT descriptors for each process are added to the GDT as the process is created.
- ★ The kernel itself spawns the first process: process running `init_task` .
- ★ During kernel initialization, the `trap_init( )` function inserts the TSS descriptor of this first process into the GDT using the statement: `set_tss_desc(0, &init_task.tss);`
- ★ The first process creates others, so that every subsequent process is the child of some existing process.
- ★ The `copy_thread( )` function, which is invoked from the `clone( )` and `fork( )` system calls to create new processes, executes the same function in order to set the TSS of the new process: `set_tss_desc(nr, &(task[nr]->tss));`
- ★ Since each TSS descriptor refers to a different process, of course, each Base field has a different value.
- ★ The `copy_thread( )` function also invokes the `set_ldt_desc( )` function in order to insert a Segment Descriptor in the GDT relative to the default LDT for the new process.

MEMORY ADDRESSING - SEGMENTATION UNIT

□ Questions

- * The segment limit contains the:
 - a) starting logical address of the process
 - b) starting physical address of the segment in memory
 - c) segment length
 - d) None of these
- * Each GDT has total of entries out of which contain segment descriptors and remaining are null, unused, or reserved entries
- * Describe the 18 segment descriptors included in each GDT.
- * Draw the figure of Global Descriptor Table showing all the descriptors and their locations (entries) in the GDT.
- * Is the following statement TRUE or FALSE
“In uniprocessor systems there is only one GDT, while in multiprocessor systems there is one GDT for every CPU in the system”.
- * State the purpose of Three Thread-Local Storage (TLS) segment descriptors in GDT.
- * State the purpose of special TSS segment Descriptor in GDT.
- * Current Privilege Level of the CPU reflects whether the processor is in User or Kernel Mode and is specified by the field of the Segment Selector stored in the cs register.
- * When the CPL is equal to 3 (User Mode), the ds register must contain the Segment Selector of the user segment, but when the CPL is equal to 0, the ds register must contain the Segment Selector of thesegment.

MEMORY ADDRESSING - SEGMENTATION UNIT

□ Questions

- * Is the following statement TRUE or FALSE "Each process descriptor includes its own TSS segment and a pointer to its LDT segment, which is also located inside the kernel data segment".
- * Draw the Segmentation Unit showing clearly all the blocks used to translate logical address to a Linear address
- * Is the following statement TRUE or FALSE
"Each process descriptor includes its own TSS segment and a pointer to its LDT segment, which is also located inside the kernel data segment."
- * Draw the Segmentation Unit showing clearly all the blocks used to translate logical address to a Linear address
- * Each GDT has total of entries out of which contain segment descriptors and remaining are null, unused, or reserved entries
- * Describe the 18 segment descriptors included in each GDT.
- * Draw the figure of Global Descriptor Table showing all the descriptors and their locations (entries) in the GDT.
- * Is the following statement TRUE or FALSE
"In uniprocessor systems there is only one GDT, while in multiprocessor systems there is one GDT for every CPU in the system."
- * State the purpose of Three Thread-Local Storage (TLS) segment descriptors in GDT.

MEMORY ADDRESSING - SEGMENTATION UNIT

□ Questions

- * State the purpose of special TSS segment Descriptor in GDT.
- * Current Privilege Level of the CPU reflects whether the processor is in User or Kernel Mode and is specified by the field of the Segment Selector stored in the cs register.
- * When the CPL is equal to 3 (User Mode), the ds register must contain the Segment Selector of the user segment, but when the CPL is equal to 0, the DS register must contain the Segment Selector of thesegment.
- * Is the following statement TRUE or FALSE
“Each process descriptor includes its own TSS segment and a pointer to its LDT segment, which is also located inside the kernel data segment.”
- * Draw the Segmentation Unit showing clearly all the blocks used to translate logical address to a Linear address.
- * Suppose the gdtr register in the processor has the value 0x00800000 and the upper 13 bits of the CS register is used as the Segment Selector and contains the value 0x0C
 - i) What is the address of the corresponding Segment Descriptor stored in the GDT (i.e, address of the corresponding GDT entry)
 - ii) If the base address stored in the segment descriptor in the GDT found at the address computed in (i) is 0x0B000000 and the register **rip** in the processor has the offset 0x100 then compute the linear address.
 - iii) What is the maximum number of Segment Descriptors that can be stored in the GDT considering a) Index bits and b) each processor GDT table

SOC 3010

OPERATING SYSTEM

END OF

WEEK 4 LECTURE 1

MEMORY ADDRESSING

SEGMENT

SOC 3010 OPERATING SYSTEMS

Reading Assignments

Chapter 2 of Text Book

Text Book

➤ Understanding the Linux Kernel

- Daniel P/ Bovet and Marco Cesati
Edition, O' Reilly Media 2005, ISBN 978-0596005658**

INHA UNIVERSITY TASHKENT

FALL SEMESTER 2024

SOC 3010

OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

